

Homework 5

Matteo Carrese

1 Assignment and objective

The assignment is to construct a Triple Graph Grammar (TGG) for transforming Use Case Diagrams into Class Diagrams with AGG. The proposed solution focuses on the main structural information of a use case model and keeps the grammar small enough to be inspected and executed step by step.

A TGG works with three connected graphs:

- the **source graph** contains the Use Case Diagram;
- the **correspondence graph** records the mappings;
- the **target graph** contains the generated Class Diagram.

The triple structure can be written as

$$S \xleftarrow{s} C \xrightarrow{t} T,$$

where S is the source model, C is the correspondence model, and T is the target model.

2 Transformation mapping

Source	Correspondence	Target
Actor	Actor2Class	Class
UseCase	UseCase2Operation	Operation
participates	Association2Ownership	owns

Table 1: The three mappings implemented by the grammar.

The value of the source **name** attribute is copied to the generated target element. As a result, an actor named **Customer** generates a class named **Customer**, while a use case named **Place order** generates an operation with the same name.

3 Triple type graph

The AGG type graph defines which nodes and edges may appear in the three domains. Source types are red, correspondence types are yellow, and target types are blue. This visual convention makes every part of the transformation easy to identify.

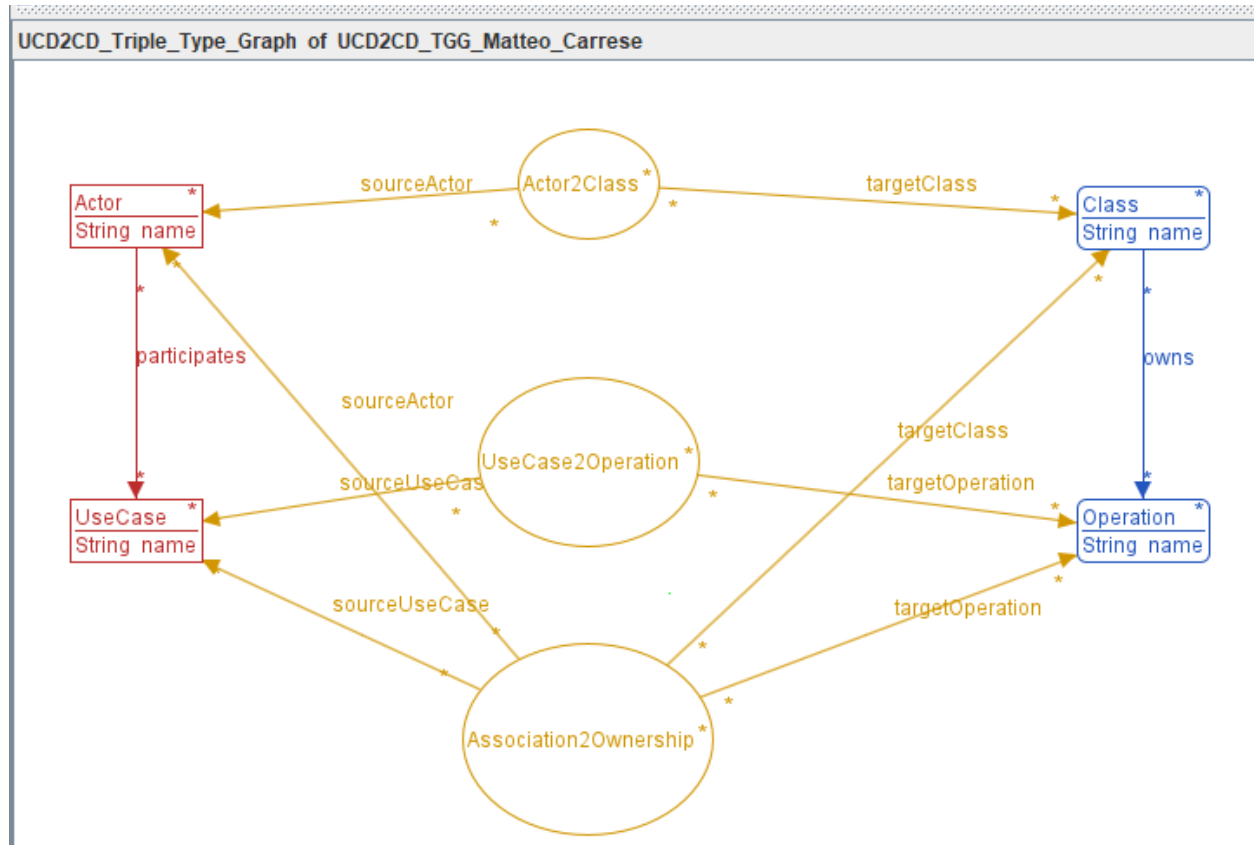


Figure 1: The complete triple type graph in AGG.

The source graph contains **Actor**, **UseCase**, and the directed edge **participates**. The target graph contains **Class**, **Operation**, and the edge **owns**. Three correspondence types connect the two sides.

The correspondence graph is not only a visual aid. It allows a rule to locate the exact target object generated from a source object. For example, the third rule follows an **Actor2Class** node instead of searching for a class only by its name.

4 Transformation rules

The grammar contains three rules executed in layer order.

4.1 ActorToClass

The first rule matches an actor and creates a class with the same name. It also creates an **Actor2Class** node connected to both objects. A Negative Application Condition (NAC) prevents the rule from running if that actor has already been transformed.

4.2 UseCaseToOperation

The second rule transforms each use case into an operation and creates a `UseCase2Operation` trace. Its NAC prevents duplicate operations.

4.3 AssociationToOwnership

The third rule is applied after the actor and use case mappings exist. Its left-hand side matches an actor, a use case, their `participates` edge, and the corresponding class and operation.

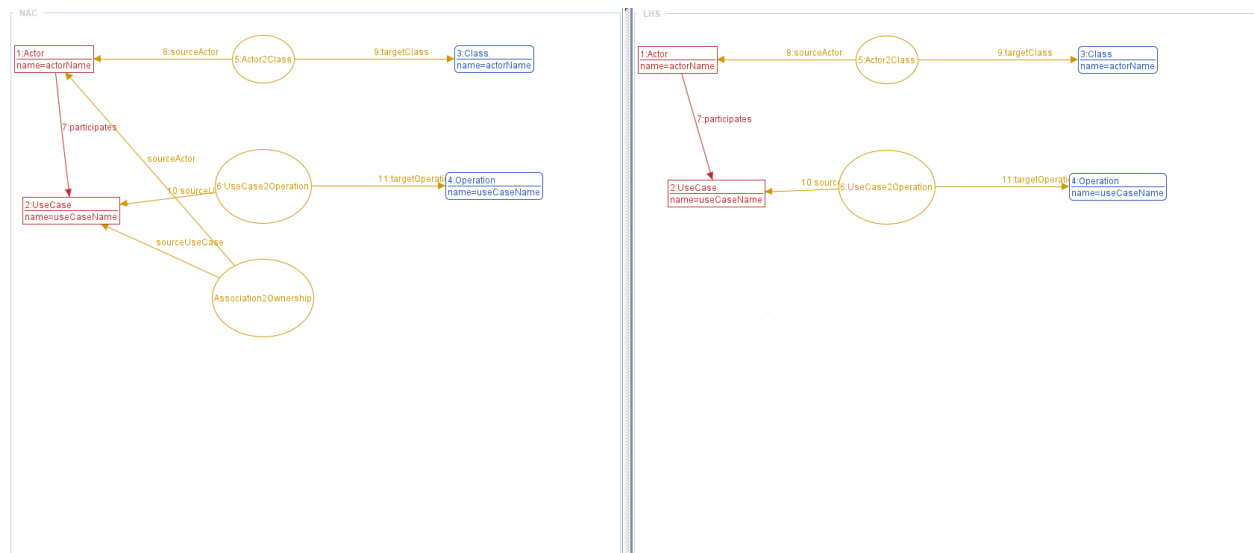


Figure 2: The NAC and left-hand side of `AssociationToOwnership`.

The right-hand side preserves all matched objects and adds an `Association2Ownership` trace together with the blue `owns` edge from the class to the operation.

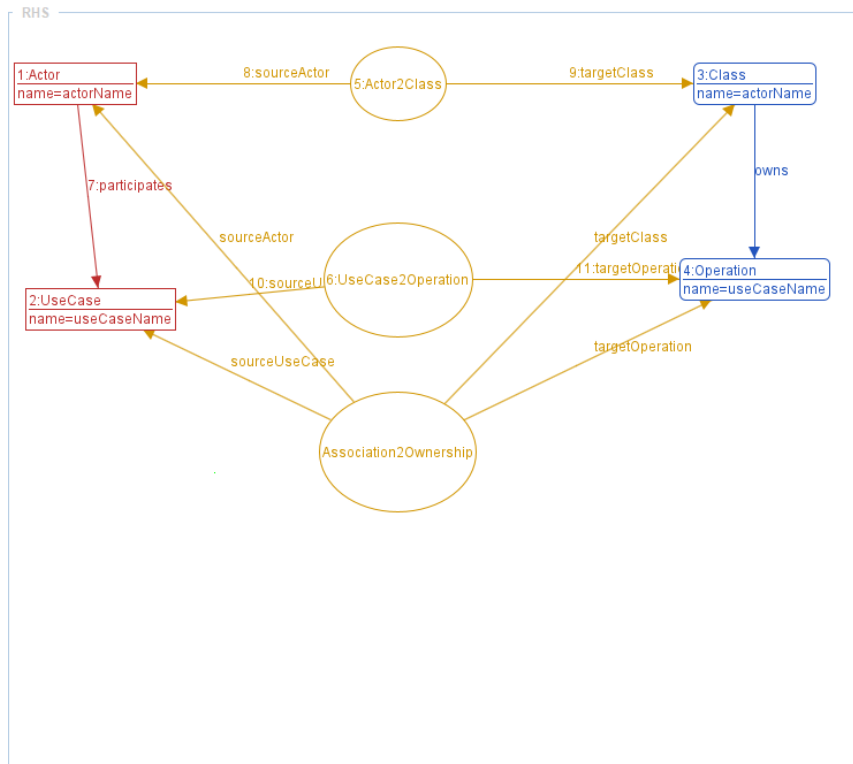


Figure 3: The right-hand side creates the trace and ownership relation.

The NAC checks that the association trace does not already exist. Therefore, the same source association cannot generate the target ownership relation more than once.

5 Example transformation

The initial host graph models a small online shop. The customer participates in **Browse catalog** and **Place order**. The administrator participates in **Manage catalog**.

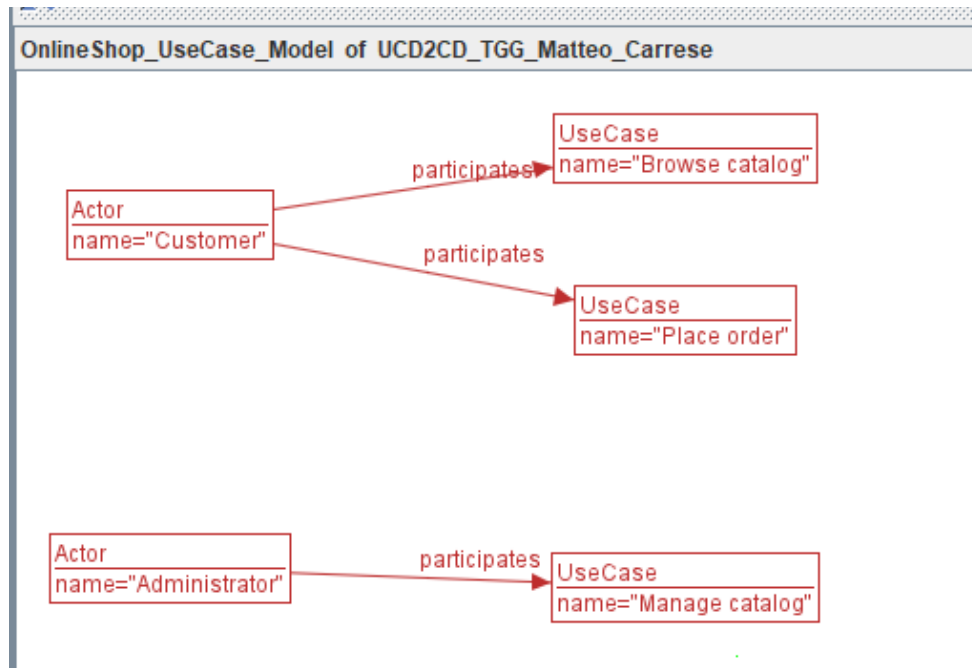


Figure 4: The source-only host graph before transformation.

AGG applies the rules by layer:

1. **ActorToClass** creates the two classes;
2. **UseCaseToOperation** creates the three operations;
3. **AssociationToOwnership** creates the three ownership edges.

The source model is preserved. The new correspondence and target elements are added to the same typed graph, allowing the complete transformation trace to be inspected.

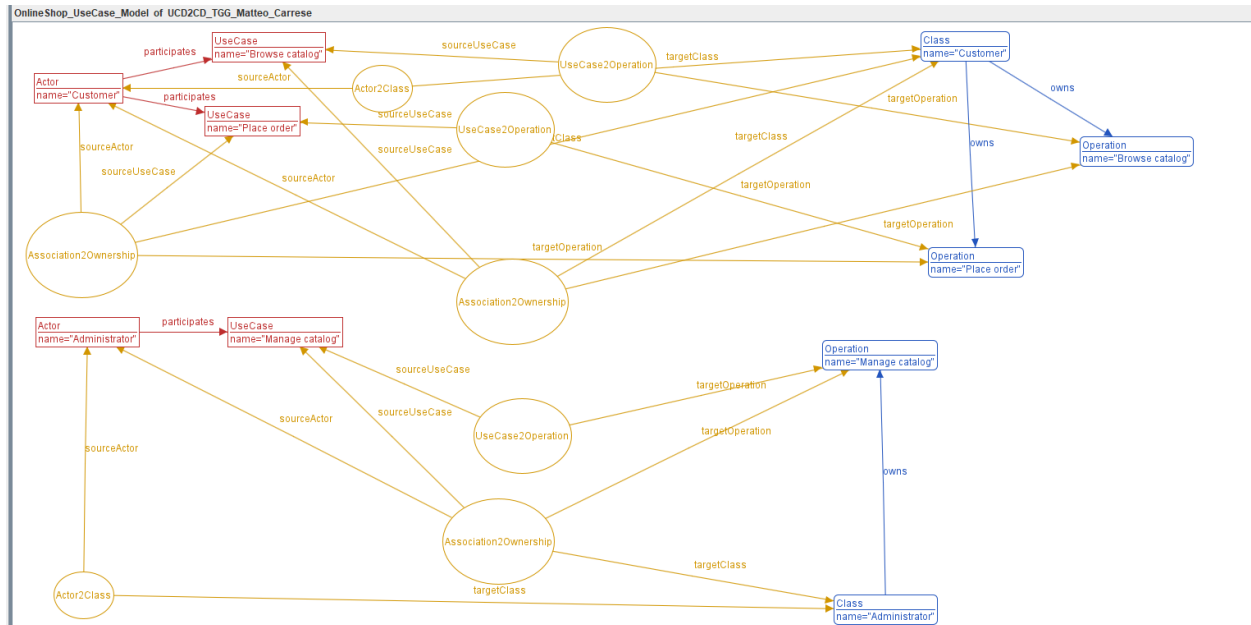


Figure 5: The host graph after the complete transformation.

The final target model contains the following information:

Generated class	Owned operations
Customer	Browse catalog; Place order
Administrator	Manage catalog

Table 2: Resulting class diagram elements.

The complete graph contains 18 nodes and 28 edges. These values include the five source nodes, three source associations, the generated target objects, all correspondence nodes, and their trace edges.

6 Termination and correctness

Each rule has one NAC. The NAC checks for the correspondence object created by that rule. Once the trace exists, the rule cannot be applied again to the same source pattern. This prevents duplicates and makes execution terminate after all source elements have been processed.

The grammar was loaded and executed with AGG. All three layers completed, and the resulting graph remained consistent with the type graph. Reapplying the transformation does not create another class, operation, or ownership relation for an already transformed element.

7 Conclusion

The grammar provides a simple but complete example of a model-to-model transformation. It separates source, correspondence, and target concepts, copies element names, preserves the original model, and prevents duplicate results through NACs. Additional UML relations such as include, extend, and generalization could be supported by adding new types and transformation rules without changing the basic three-domain architecture.